

Scaling Mobile Development Instruction Through Tooling: A Teaching-Assistant Case Study in CS 5520

Kaustubha Venkata Eluri

Teaching Assistant – CS 5520 Mobile Application Development
Northeastern University, Khoury College of Computer Sciences
San Jose, CA, USA

Abstract—This paper documents pedagogical engineering work performed as a Teaching Assistant for CS 5520 (Mobile Application Development) at Northeastern University’s Khoury College of Computer Sciences. The work covered mentoring 30+ graduate students on Android development, building Python-based auto-graders to scale feedback delivery, leading weekly labs, and redesigning course assignments in collaboration with faculty to better reflect industry mobile-development patterns. We report the measured outcomes of these interventions – assignment score improvement, grading-time reduction, project completion rate, and assignment completion-rate improvement – and discuss the underlying premise that tooling investment, not just additional instructor time, is what allows a single teaching assistant to scale feedback quality across a large graduate cohort.

Index Terms—computer science education, teaching tools, automated grading, mobile development pedagogy, Android, MVVM

I. INTRODUCTION

A single teaching assistant supporting 30 or more graduate students in a hands-on mobile development course faces a scaling problem: manual code review and personalized feedback do not scale linearly with headcount, and the quality of feedback a student receives should not depend on how many other students happen to have submitted at the same time. This motivates treating course support itself as an engineering problem: where can tooling substitute for, or amplify, a single TA’s manual review capacity without degrading feedback quality.

II. ROLE AND SCOPE

As the Teaching Assistant for CS 5520, Mobile Application Development, the author mentored 30+ graduate students on Android development using Java, Firebase, and MVVM architecture, built grading infrastructure, led weekly labs, and worked directly with course faculty on assignment redesign.

III. AUTO-GRADERS AS A SCALING MECHANISM

The author built Python-based auto-graders and structured grading rubrics for course assignments, reducing grading time by 40% and standardizing feedback turnaround to within 48 hours. Automated grading for a mobile-development course is nontrivial relative to, for example, autograding a pure-algorithms assignment: Android assignments typically require

build verification, UI-state checks, and adherence to architectural patterns (e.g., MVVM layering) that a naive test-only autograder does not capture well. Structured rubrics paired with automation allowed grading time to fall by 40% while keeping feedback turnaround bounded at 48 hours regardless of submission volume, which is the property that matters most to students: a predictable, bounded feedback loop rather than a variable one dependent on TA availability.

IV. WEEKLY LABS AND DIRECT MENTORSHIP

Beyond grading infrastructure, the author led weekly labs covering the Android lifecycle, networking, and local storage – the three areas that, in the author’s experience running the labs, produced the highest volume of student debugging questions in a Java-based Android course. Ninety percent of students shipped fully functional final projects, and mentoring 30+ graduate students contributed to an average 15% improvement in assignment scores across the cohort. A 90% final-project completion rate in a mobile-development course is a meaningful outcome given that final mobile projects typically integrate several previously separate course concepts (persistence, networking, lifecycle-aware UI) simultaneously, and integration failures at that stage are common in courses without strong incremental lab support.

V. ASSIGNMENT REDESIGN IN COLLABORATION WITH FACULTY

The author worked with course faculty to redesign two major assignments to better reflect current industry mobile-development patterns – specifically, offline-first design and push notifications – rather than leaving the assignments as originally scoped. This redesign increased assignment completion rates by 12 percentage points. Aligning course assignments with patterns students are likely to encounter in industry (offline-first data handling, push-notification integration) rather than with simplified academic exercises appears, based on this completion-rate improvement, to have increased student engagement with the assignments rather than decreasing it, despite the added realism typically increasing an assignment’s apparent complexity.

VI. DISCUSSION

This engagement illustrates a general principle in scaling technical teaching support: the constraint is rarely a TA's technical knowledge of the subject matter, but the TA's available hours relative to cohort size. Auto-grading infrastructure and structured rubrics convert a linear-in-headcount feedback cost into a largely fixed cost (build the autograder once, apply it to all submissions), freeing the TA's remaining hours for the direct mentorship and lab instruction that automation cannot replace. The faculty-collaborative assignment redesign further indicates that course content quality and instructional-support scaling are separate levers – both were exercised in this engagement, and the completion-rate and score improvements reflect their combined effect rather than either lever in isolation.

VII. LIMITATIONS

The metrics reported here (15% score improvement, 40% grading-time reduction, 90% project completion, 12-point completion-rate increase) are drawn from the author's own record of the CS 5520 offering(s) supported during this engagement, and are not benchmarked against a controlled comparison cohort that did not receive the same tooling and redesign interventions; the reported improvements should therefore be read as outcomes observed within this offering rather than as causally isolated effects of any single intervention.

VIII. CONCLUSION

This case study documents how auto-grading tooling, structured lab instruction, and faculty-collaborative assignment redesign combined to support a 30+ student graduate cohort in a hands-on Android development course, yielding a 15% average score improvement, a 40% reduction in grading time with a bounded 48-hour feedback turnaround, a 90% final-project completion rate, and a 12-percentage-point increase in assignment completion following an industry-aligned redesign.